From instructions to intent: working agentically with AI

- AI and you
  - Context matters
- From copy-🍝 to agentic AI
  - Width vs depth
    - Width – freedom of interpretation
    - Dept – autonomy over time
  - Workflows
    - Prompt engineering
    - AI Agents
    - Spec-Driven Development (INFO is moving towards here)
    - Agentic AI
  - TL;DR
    - Picking the correct workflow
- What this all means
  - Shift up
  - Skills
  - Risks

Before we dive into it, it is good to be familiar with the following terms

term	meaning
Artificial intelligence (AI)	AI is the ability of computer systems to perform tasks that normally require human intelligence, such as learning, reasoning, and decision-making. What Is Artificial Intelligence (AI)? | IBM
Large Language Models (LLMs)	LLMs are AI systems trained on large amounts of text to understand and generate *human-like* language. What Are Large Language Models (LLMs)? | IBM
Models	An AI model is a trained program that recognizes patterns in data to make predictions or generate outputs. What Is an AI Model? | IBM
Agents	AI agents are autonomous systems that use AI models to perform tasks and make decisions with little or no human supervision. What Are AI Agents? | IBM
Prompt	A prompt is an instruction or input given to an AI to elicit a specific response or action.

	What Is Prompt Engineering? | IBM

AI and you

AI is ~~going to be~~ already transforming the way we work.

This page is dedicated to showcase the different workflows, from instructions (augmentation) to intent (automation), in using LLMs in your day-to-day tasks.

As a digital agency, we should be on the forefront of exploring and experimenting to bring business value using these tools before businesses (can) do it themselves.

You can utilize your Personal Development Time to deepen your understanding of AI, build practical skills, and explore how these tools can support your work.

> ⚠ This page is a living document. Expect it to evolve as our tools, workflows, and understanding (both from the industry and from explorations done by **you**) improve.

> ℹ Some resources talk about "[...] in coding"; however, these resources add value to **all** disciplines, as the understanding of the workflows and techniques is *fundamental*.

Context matters

Pun intended.

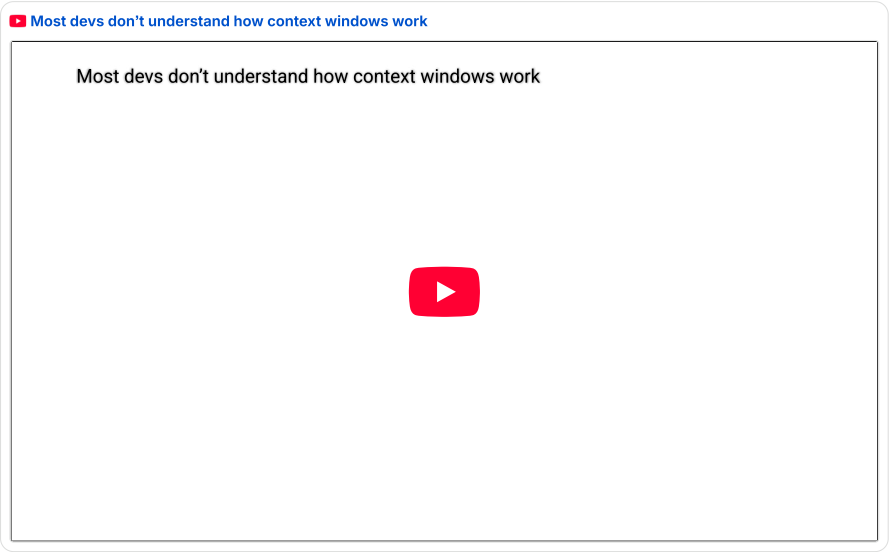
To be able to work efficiently with this all, regardless of your workflow, Attention Is All You Need to work better with(in) the `context (window)` of AI.

Each model can only operate in a certain a pre-defined *conversation size* called the `context window`, measured in tokens. You (read INFO) pay for this token usage, see LLM Token Limits: Every Model's Context Window Compared (2026) | Morph.

Managing context, what to put in and what to omit is a balancing game between signal and noise.

Down the rabbit hole

- Using CLAUDE.MD files: Customizing Claude Code for your codebase | Claude
- Memory - OpenClaw
- Context windows



From copy-🍝 to agentic AI

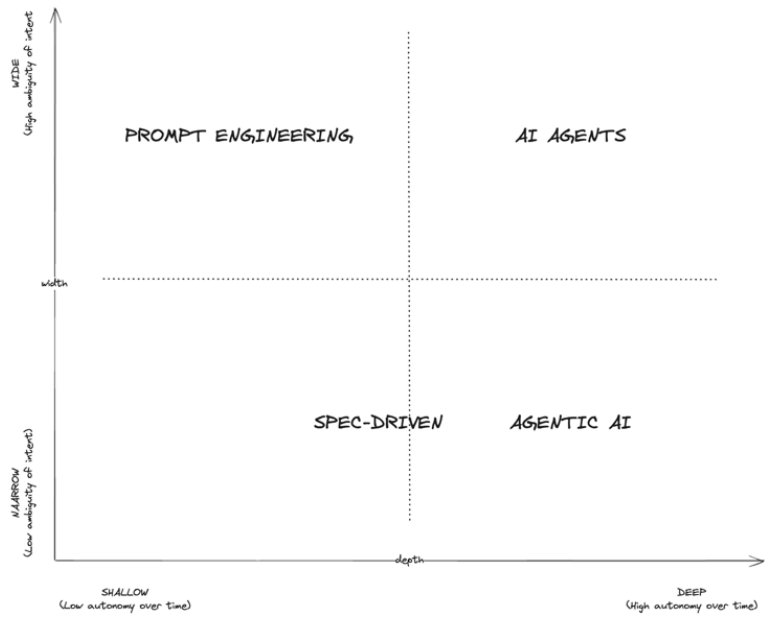
There is no **one** correct way to work with AI <period>

Instead, there is a range of workflows, each with different strengths, risks and “ideal” use cases. Understanding these different workflows can help you pick the right approach for the task at hand.

Width vs depth

Each workflow can be seen from two *independent* properties: **width** and **depth**.

These two dimensions explain why some workflows feel powerful, others feel frustrating, and some feel outright dangerous when used in the wrong situation.



Where each workflow “sits”

Width – freedom of interpretation

> How much ambiguity is allowed and how many valid paths exist at any moment.

Wide workflow	Narrow workflow
Many acceptable interpretations	Few valid interpretations
Vague or flexible intent	Intent is explicit and constrained
AI has room to “fill in the blanks”	AI freedom is bounded
Humans correct *after the fact*	Humans correct *before execution*

Dept – autonomy over time

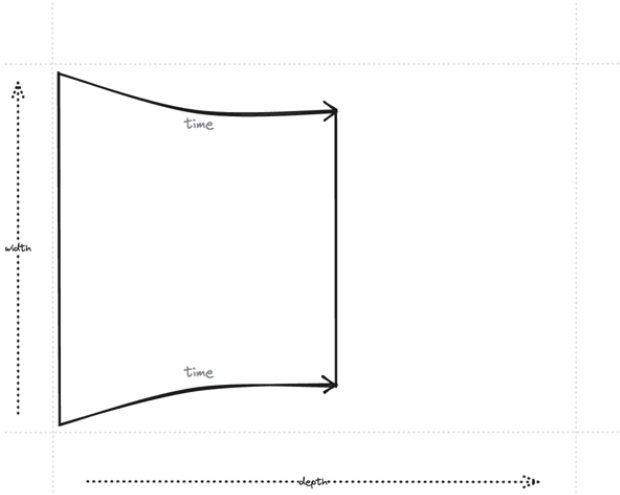
> how many steps the AI can take on its own *before* a human intervenes.

Shallow workflow	Deep workflow
One-shot or near one-shot	Multistep planning and execution

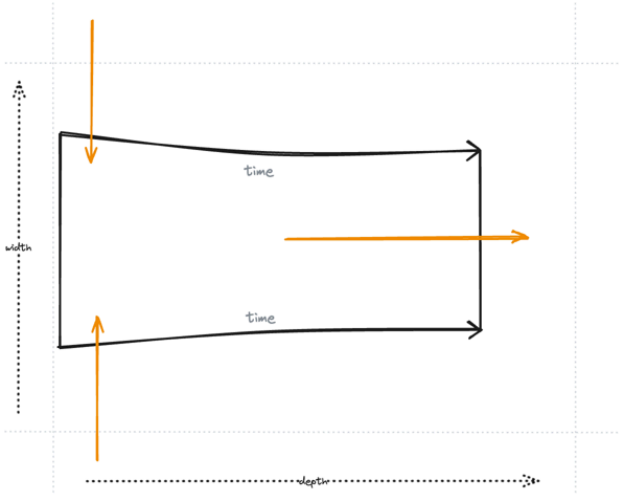
Human involved at every step	Human steps in later or only on escalation, intermediate reasoning may be hidden
Errors are easy to spot quickly	Errors can compound silently

> Depth is not a goal by itself. Depth should be earned through narrowness. The more freedom you give an AI over time, the clearer your intent and constraints must be upfront.
>
> *E.g., Spec-driven workflows often start narrow and shallow, and only become deep once intent, constraints, and validation are strong enough to support autonomy.*

"Depth should be earned through narrowness." — visualized



Starting `WIDE` only gets you so far, a `SHALLOW` depth

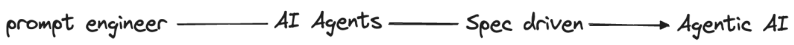


By `NARROW`ing the width, you allow for a `DEEP`er autonomy over time

Workflows

At INFO, we are exploring how we can move towards working in a **spec driven** way of working with(in) teams and where/how AI can support this new workflow across expertise.

Each workflow is an abstraction on top of the previous one, to better work with the latter it is good to (fully) understand the former.



4 distinct workflows of working with AI

Prompt engineering

> AI as a smarter search engine or autocomplete

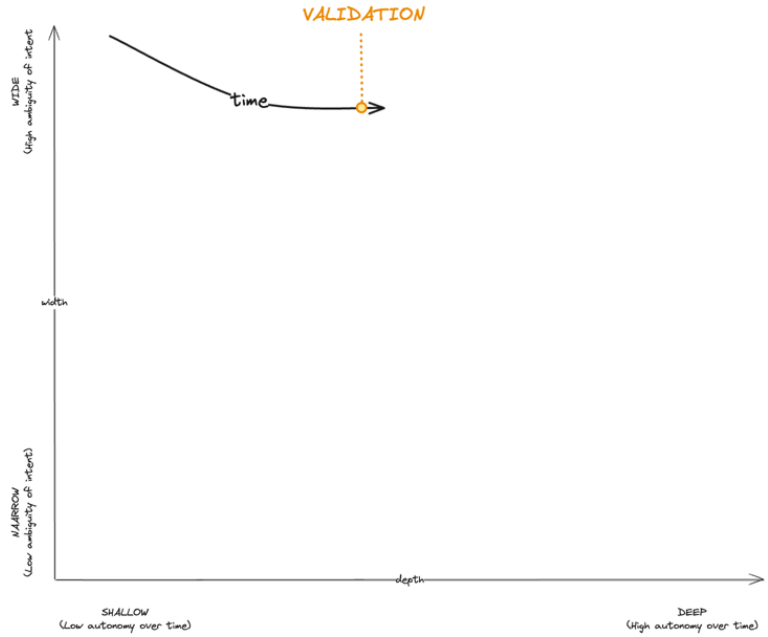
Prompt engineering refers to the methodology on how early interactions started to form with the release of LLMs such as ChatGPT back in `1 Nov 2022`. By *engineering* the *prompts,* you can better steer the output that is produced.

Although this is the first workflow, many tasks *should* still live in this space. Here, you collaboratively work with *LLMs* to make *something*. You prompt, AI responds, you copy, paste and adapt the output. You are the one who integrates everything manually.

This workflow works well for exploring ideas, getting unstuck, **drafting** pieces of text or even learning unfamiliar domains. It is (usually) very fast, has a low cognitive overhead, is easy to adopt and helps to improve your individual productivity. On the other hand, the *context* is short-lived and the quality of output heavily depends on your *prompting*.

This workflow breaks down when tasks grow larger, require shared understanding, or span multiple steps.

> ⚠ **Key risk**
> Mistaking good wording for good thinking.



`WIDE` & `SHALLOW`

Common tooling

- ChatGPT
- Microsoft Copilot: Your AI companion
- Claude Code by Anthropic | AI Coding Agent, Terminal, IDE
- Ollama + AI your way. Simple. Powerful. Private.

Down the rabbit hole

- The Prompt Report: A Systematic Survey of Prompt Engineering Techniques
- GitHub - danielmiessler/Fabric: Fabric is an open-source framework for augmenting humans using AI. It provides a modular system for solving specific problems using a crowdsourced set of AI prompts that can be used anywhere.

Most devs don't understand how LLM tokens work

AI Agents

> AI as an assistant that can take multiple steps

Agents are ways to give your LLMs a bit more autonomy. Here, AI starts doing more than answering a question. It can plan, act, and use tools. This is where your LLM starts to get integrated in/with the tooling you already use.

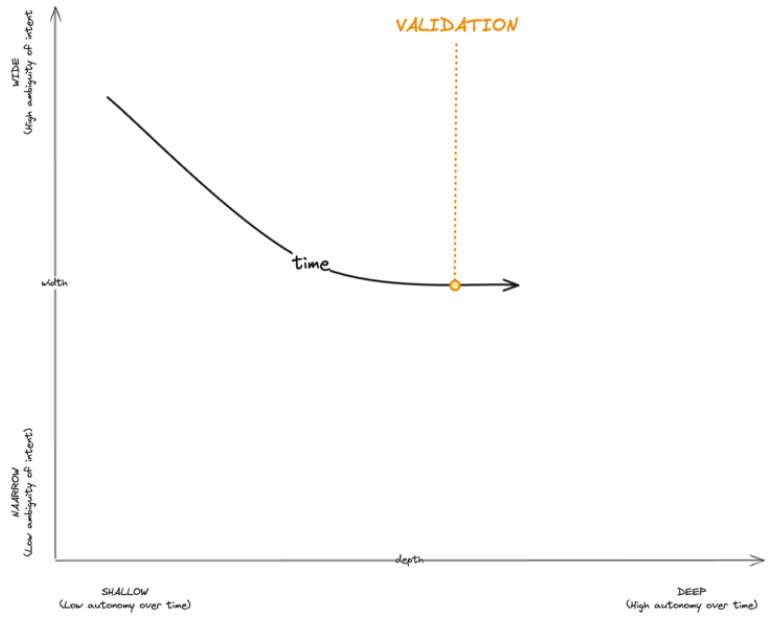
You describe a task or outcome, AI determines intermediate steps. It may write code, query data, or iterate, and you review the final result.

This workflow works well for research and synthesis, code scaffolding, refactoring or analysis, or tasks with a clear goal but a flexible path. It can handle multistep problems, reduces manual *"busywork"*, explore solution spaces quickly. This should feel more like delegating a task.

On the downside, intermediate reasoning and decisions are not always visible, goals and success criteria are often inferred rather than explicit and mistakes can propagate and compound quietly across steps without being noticed.

This workflow breaks down when there is no clear *intent*. Without clear intent, agents optimize for *something* — just not always the right thing.

> ⚠ **Key risk**
> The AI sounds confident even when the goal was under-specified.



`WIDE` & `DEEP`

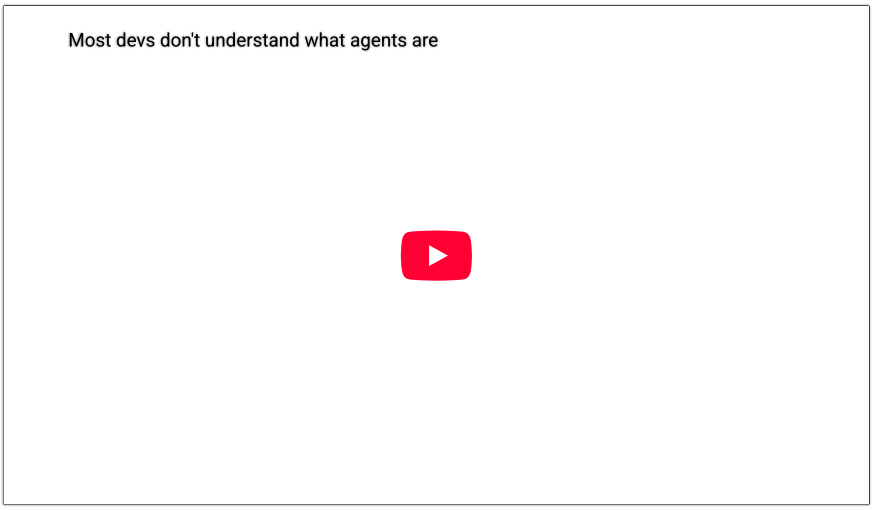
Common tooling

- GitHub Copilot in VS Code
- GitHub - modelcontextprotocol/servers: Model Context Protocol Servers
- Skills | Claude / Connect Claude Code to tools via MCP - Claude Code Docs

Down the rabbit hole

- Evaluating AGENTS.md: Are Repository-Level Context Files Helpful...

Most devs don't understand what agents are



Spec-Driven Development ***(INFO is moving towards here)***

> AI as an executor of clearly defined intent

This workflow is where AI stops being “just helpful” and starts becoming (a) reliable (team member). In this workflow, you are only in the driver's seat by thoroughly describing your intent, after this AI takes over.

You write specifications (spec); goals, constraints, assumptions, success criteria. AI executes against this spec and the output is automatically reviewed against the same spec. Most thinking happens before execution.

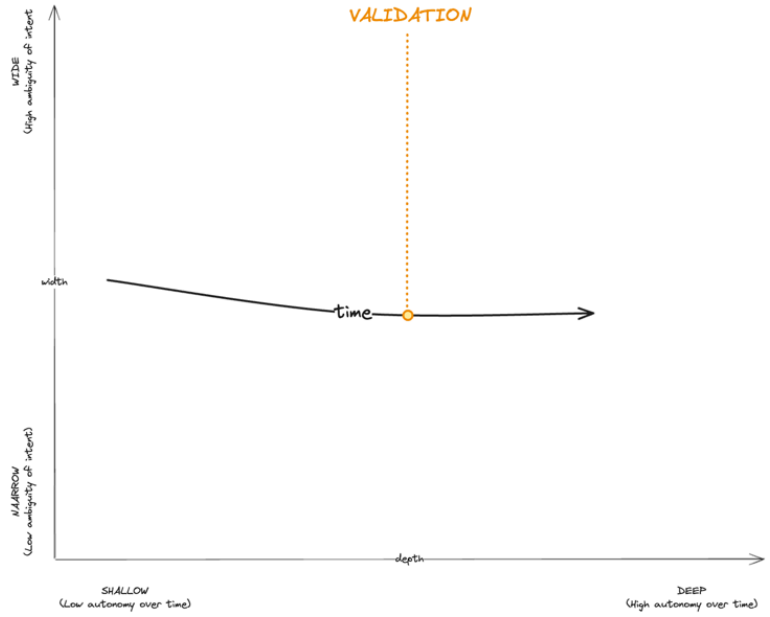
Specs are opinionated pieces of text, often presented in `Markdown`, and become shared artifacts. When done well, specs create a shared understanding across roles and disciplines. Ambiguity is no longer hidden inside prompts or intermediate steps, but surfaced early and made discussable. Both between humans, and between humans and AI.

This works well where correctness matters more than speed and in cross-team/cross-discipline collaboration settings. Creating a shared understanding between humans and AI where ambiguity surfaces early. Outputs are more consistent, and are easier to review and iterate on.

This workflow has a higher upfront “cost”, in time and tokens, requires more discipline, a better domain understanding and expertise of your field of work as well as knowing what you *do not* want as much as what you do. All of this typically feels heavier at first, but pays off rapidly as complexity grows, reducing misalignment and rework.

All of this should not sound foreign when working in an Agile environment.

> ⚠ **Key risk**
> Specs have a way of enhancing both mastery and misunderstanding. They encode what we believe to be true, including the blind spots we may not (yet) see.



`NARROW` & `SHALLOW` or `DEEP`

Common tooling

- GitHub - github/spec-kit: 💫 Toolkit to help you get started with Spec-Driven Development
- GitHub - gsd-build/get-shit-done: A light-weight and powerful meta-prompting, context engineering and spec-driven development system for Claude Code by TÂCHES.

- Plan agent - CLI - Docs - Kiro
- https://code.visualstudio.com/docs/copilot/agents/planning

I was an AI skeptic. Then I tried plan mode

Agentic AI

> AI as a collaborator inside a designed system

Agentic AI is not about giving an AI model full freedom. It is about designing systems in which AI can operate autonomously within clearly defined boundaries. This workflow shifts effort: less time spent on doing, more time spent on designing how work should happen.

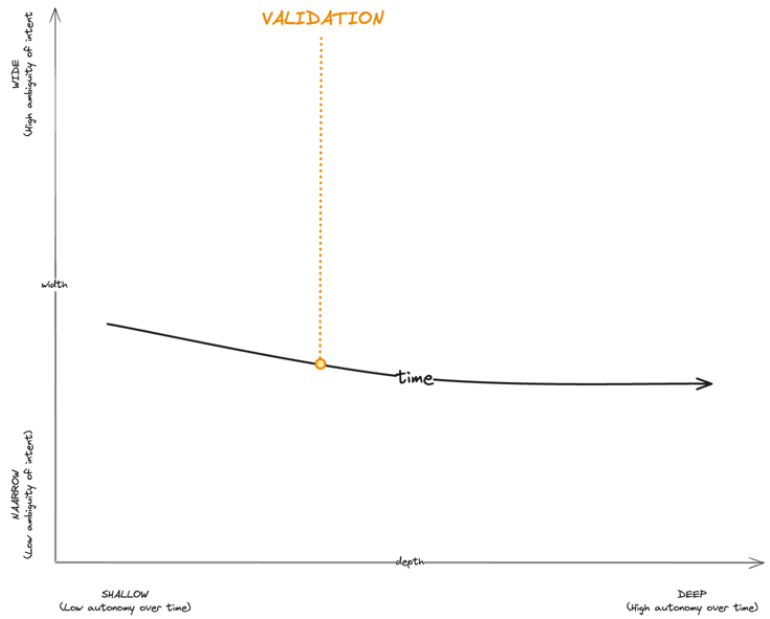
In this workflow, AI systems are capable of planning work, executing tasks, evaluating their outputs, and iterating, all while humans remain responsible for intent, constraints, and decision-making. Rather than prompting or even delegating individual tasks, you design a *system* in which AI can repeatedly perform work.

This works well for repetitive but complex workflows, integrations across tools, data sources or services and situations where *process consistency* (e.g., continuous documentation maintenance) matters more than individual output.

The problems with this workflow become evident when constraints are unclear or incomplete, human oversight is implicit instead of designed, or responsibility is shifted away instead of upward.

Key risk

Confusing autonomy with accountability.



NARROW & DEEP

Common tooling

- n8n.io - AI workflow automation tool
- Ralph Wiggum - Viral Agentic Coding Loop, Simplified

Down the rabbit hole

- GitHub - rowboatlabs/rowboat: Open-source AI coworker, with memory
- OpenClaw — Personal AI Assistant / GitHub - qwibitai/nanoclaw: A lightweight alternative to OpenClaw that runs in containers for security. Connects to WhatsApp, Telegram, Slack, Discord, Gmail and other messaging apps,, has memory, scheduled jobs, and runs directly on Anthropic's Agents SDK
- GitHub - snarktank/antfarm: Build your agent team in OpenClaw with one command.

Ship working code while you sleep with the Ralph Wiggum technique

TL;DR

If a workflow feels frustrating, it is often a sign you are using it beyond its natural limits. Not that you are “doing AI wrong”.

Picking the correct workflow

These tables are for picking the “correct”* workflow at hand, looking at it from different angles

* Your mileage might vary, explore and experiment to find the right workflow **for you**.

What do you want to achieve

Situation	or **Goal**	**Recommended workflow**
Exploring ideas	Learn & explore	Prompt-based
One-off tasks	Get it done	Prompt-based or Agent
Complex execution	Reduce risk	Agent or Spec driven
Shared systems	Align teams	Spec driven
Repeated workflows	Scale work	Agentic

Who should have the agency

	Prompt engineering	**AI Agents**	**Spec driven**	**Agentic AI**
Human (you)	Express intent via prompts	Define tasks & outcomes	Define intent, constraints & success criteria	Design systems, rules & guardrails
AI autonomy	Single response per prompt	Plans & executes (multistep) tasks	Executes against explicit specs	Plans, acts, evaluates & iterates
Validation mechanism	Human manually selects & integrates the correct parts	Human reviews output	Spec-based validation	Systematic validation with human escalation

AI may gain autonomy, but responsibility and accountability always remain with humans.

What this all means

Cool, but what does this all mean (to me)?

It is good to note that AI does not replace thinking nor removes responsibilities.

What matters more than picking “the right tool” or “the best AI” is how we stay in control and the processes we put in place to work collaboratively and safely with AI.

Shift up

We move from doing, formatting and translating towards framing problems, setting constraints, reviewing outcomes and making decisions.

Your way of working should “shift up” to a higher abstraction level.

Skills

Some skills defined in 📚 The Structural skills matrix translate very precisely into this new way of working

- Writing clear intents; goals, constraints, and success criteria translate well to the *Communication* and *Collaboration*.
- Systematically build and utilize various workflows for different tasks goes hand in hand with the skills described in *System thinking* and *Strategic thinking*.
- Managing the right context and critically reviewing outcomes works well with *Time ownership* and *Learning Agility*.

Risks

The 🐘 in the room.

While we can move a lot of work towards AI, having a deep and systematic understanding of your work should still, and will always, be the baseline*.

Over-delegation can lead to imbalance, (hallucinated) confidence can lead to brittle systems and context loss can lead to the losing of (shared) understanding (in teams).

Down the rabbit hole

Why I stopped using AI code editors ·Luciano Nooijen

AI is Creating a Generation of Illiterate Programmers

* what you need to know drifts over time, though.